

Paradigmas de Programação — Aula 08:

Concorrência, gorrotinas e canais

Processos, threads e condição de corrida

Processo e thread

- **Processo**: programa em execução com **espaço de endereçamento próprio**
 - a memória de um não é visível para outro
 - comunicação por meio externo: *pipe*, *socket*, arquivo, saída padrão
- **Thread**: linha de execução **dentro** de um processo
 - pilha própria, mas **memória compartilhada** com as outras threads
 - comunicação barata: escrever numa variável que a outra lê

`01-processo.java` : o filho incrementa `contador` e imprime 11; o pai continua com 10.

A ordem não é do programa

- Quem decide qual thread executa, por quanto tempo e em que núcleo é o **escalonador**
- A decisão varia entre execuções, conforme a carga da máquina
- `02-threads.java` , duas execuções seguidas:

```
B 1 A 1 B 2 B 3 B 4 A 2 B 5 A 3 A 4 A 5  
A 1 B 1 A 2 B 2 B 3 A 3 A 4 B 4 B 5 A 5
```

Condição de corrida

- **Condição de corrida** (*race condition*): o resultado depende da ordem em que operações concorrentes sobre um dado compartilhado são intercaladas
- `contador++` são três passos: **ler**, **somar 1**, **escrever**

```
Runnable incrementar = () -> {  
    for (int i = 0; i < 100_000; i++) {  
        contador++;  
    }  
};
```

- Duas threads, 100 000 incrementos cada: esperado **200 000**
- Três execuções: **124 129**, **122 551**, **117 646**

Onde o incremento se perde

Passo	Thread A	Thread B	contador
1	lê 5		5
2		lê 5	5
3	soma: 6		5
4		soma: 6	5
5	escreve 6		6
6		escreve 6	6

Dois incrementos, um avanço: as duas leram o mesmo valor antes de qualquer uma escrever.

Exclusão mútua com `synchronized`

- **Seção crítica:** o trecho que acessa o dado compartilhado
- **Exclusão mútua:** no máximo uma thread executa a seção crítica por vez

```
static synchronized void incrementar() {  
    contador++;  
}
```

- `04-com-trava.java` : sempre **200 000**
- **Custo:** a seção crítica é serializada — enquanto uma thread a executa, as outras esperam
- A trava é responsabilidade do programador: sem ela, o programa compila e erra só em algumas execuções